

LABORATORIO N° 2

Asignatura: ANÁLISIS NUMÉRICO, Repetición Escuela: INFORMÁTICA Ciclo: III

Vasquez Daza Wagner Jesus Semestre 2026-II Semana: 3

1.Enunciado

Implemente en OCTAVE el algoritmo correspondiente al método de bisección estudiado, primero como un script y luego como una función. Haga las pruebas correspondientes aplicándolo a cada uno de los siguientes ejercicios.

Código en Octave, versión script

```
lc; clear;

% =====
% ENTRADA
% =====
f_str = input('Ingrese la función f(x): ', 's');
f = str2func(['@(x) ' f_str]);
a = input('Ingrese el extremo a: ');
b = input('Ingrese el extremo b: ');
TOL = input('Ingrese la tolerancia TOL: ');
N0 = input('Ingrese el número máximo de iteraciones N0: ');
% =====
% Paso 1
i = 1;
FA = f(a);
% Paso 2
while i <= N0
    % Paso 3
    p = a + (b - a) / 2;
    FP = f(p);

    % Paso 4
    if FP == 0 || (b - a) / 2 < TOL
        fprintf('Procedimiento completado exitosamente tras %d iteraciones.\n', i);
        fprintf('Raíz encontrada p = %f\n', p);
        return;
    end
    % Paso 5
    i = i + 1;

    % Paso 6
    if FA * FP > 0
        a = p;
        FA = FP;
    else
        b = p;
    end
end
fprintf('El método fracasó después de %d iteraciones N0 = %d\n', N0, N0);
```

Código en Octave, versión función

```
function p = biseccion(f, a, b, TOL, N0)

    i = 1;
    FA = f(a);

    while i <= N0

        p = a + (b - a) / 2;
        FP = f(p);

        if FP == 0 || (b - a) / 2 < TOL
            fprintf('Procedimiento completado exitosamente tras %d iteraciones.\n', i);
            fprintf('Raíz encontrada p = %f\n', p);
            return;
        end

        i = i + 1;

        if FA * FP > 0
            a = p;
            FA = FP;
        else
            b = p;
        end
    end

    fprintf('El método fracasó después de %d iteraciones, N0 = %d\n', N0, N0);
    p = NaN;
end
```

2. Enunciado

Determine las raíces reales de $f(x) = -\pi^{0.3}x^2 - \frac{7}{3}x + 0.55e$

a) Gráficamente

b) Empleando la fórmula cuadrática

c) Use el método de bisección y un número de iteraciones suficiente para determinar la raíz más grande de la función dada, con una tolerancia para el error absoluto de 5×10^{-6} . Considere el dominio de la función el intervalo $[a,b] = [-4,3]$. Construya una tabla que muestre cada uno de los resultados parciales, el error relativo porcentual y el error absoluto para cada iteración.

Código en Octave

```
% Problema 2
f = @(x) -(pi^0.3).*x.^2 - (7/3).*x + 0.55*exp(1);
% b: formula cuadratica
a_coef = -(pi^0.3); b_coef = -7/3; c_coef = 0.55*exp(1);
disc = b_coef^2 - 4*a_coef*c_coef;
r1 = (-b_coef + sqrt(disc)) / (2*a_coef);
r2 = (-b_coef - sqrt(disc)) / (2*a_coef);
fprintf('Raices por formula cuadratica: r1 = %.6f, r2 = %.6f\n', r1, r2);

% a: grafica
x = linspace(-4, 3, 400);
figure; hold on;
yline(0, 'k');
xlabel('x'); ylabel('f(x)'); grid on;
title('f(x) = -\pi^{0.3} x^2 - 7/3 x + 0.55e');

h = plot(NaN, NaN, 'b', 'LineWidth', 1.5);
for k = 1:length(x)
    set(h, 'XData', x(1:k), 'YData', f(x(1:k)));
    pause(0.01);
end

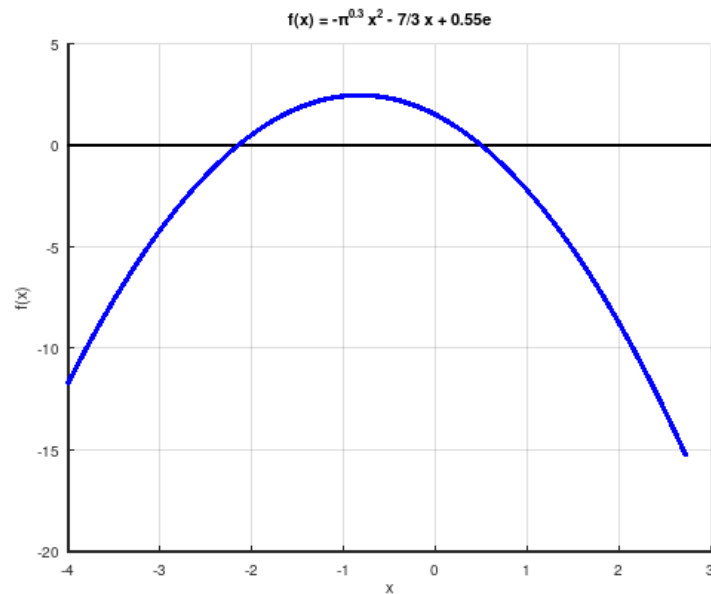
% c: biseccion
a = 0; b = 2; TOL = 5e-6;
FA = f(a); p_prev = NaN; i = 1;
fprintf(' i |    a    |    b    |    p    |    f(p)    | Ea %% \n');
while true
    p = a + (b-a)/2;
    FP = f(p);
    if isnan(p_prev)
        ea = NaN;
    else
        ea = abs((p-p_prev)/p)*100;
    end
    fprintf('%2d | %8.5f | %8.5f | %11.8f | %11.4e | %7.4f\n', i, a, b, p, FP, ea);
    if FP == 0 || (b-a)/2 < TOL
        break;
    end
    p_prev = p; i = i + 1;
    if FA*FP > 0
```

```

        a = p; FA = FP;
    else
        b = p;
    end
end
fprintf('Raiz por biseccion: p = %.6f, en %d iteraciones\n', p, i);

```

Resultados de ejecución



Raices por formula cuadratica: r1 = -2.148688, r2 = 0.493559

i	a	b	p	f(p)	Ea %
1	0.00000	2.00000	1.00000000	-2.2480e+00	NaN
2	0.00000	1.00000	0.50000000	-2.4051e-02	100.0000
3	0.00000	0.50000	0.25000000	8.2361e-01	100.0000
4	0.25000	0.50000	0.37500000	4.2181e-01	33.3333
5	0.37500	0.50000	0.43750000	2.0438e-01	14.2857
6	0.43750	0.50000	0.46875000	9.1543e-02	6.6667
7	0.46875	0.50000	0.48437500	3.4090e-02	3.2258
8	0.48438	0.50000	0.49218750	5.1054e-03	1.5873
9	0.49219	0.50000	0.49609375	-9.4515e-03	0.7874
10	0.49219	0.49609	0.49414062	-2.1677e-03	0.3953
11	0.49219	0.49414	0.49316406	1.4702e-03	0.1980
12	0.49316	0.49414	0.49365234	-3.4842e-04	0.0989
13	0.49316	0.49365	0.49340820	5.6097e-04	0.0495
14	0.49341	0.49365	0.49353027	1.0630e-04	0.0247
15	0.49353	0.49365	0.49359131	-1.2106e-04	0.0124
16	0.49353	0.49359	0.49356079	-7.3786e-06	0.0062
17	0.49353	0.49356	0.49354553	4.9459e-05	0.0031
18	0.49355	0.49356	0.49355316	2.1040e-05	0.0015
19	0.49355	0.49356	0.49355698	6.8309e-06	0.0008

Raiz por biseccion: p = 0.493557, en 19 iteraciones

3.Enunciado

Determine las raíces reales de $2x\cos(2x) - (x+1)^2 = 0$:

a) Gráficamente;

b) utilizando el método de bisección, para los valores iniciales $a = -3$ y $b = -2$ o $a = -1$ y $b = 0$, iterando hasta que el error relativo porcentual se encuentre debajo de 0.005%.

Nota: la tolerancia para el error relativo, para un número de cifras significativas deseado es dado por $es = (0.5 \times 10^{(2-n)})\%$, donde n es el número de cifras significativas.

Código en Octave

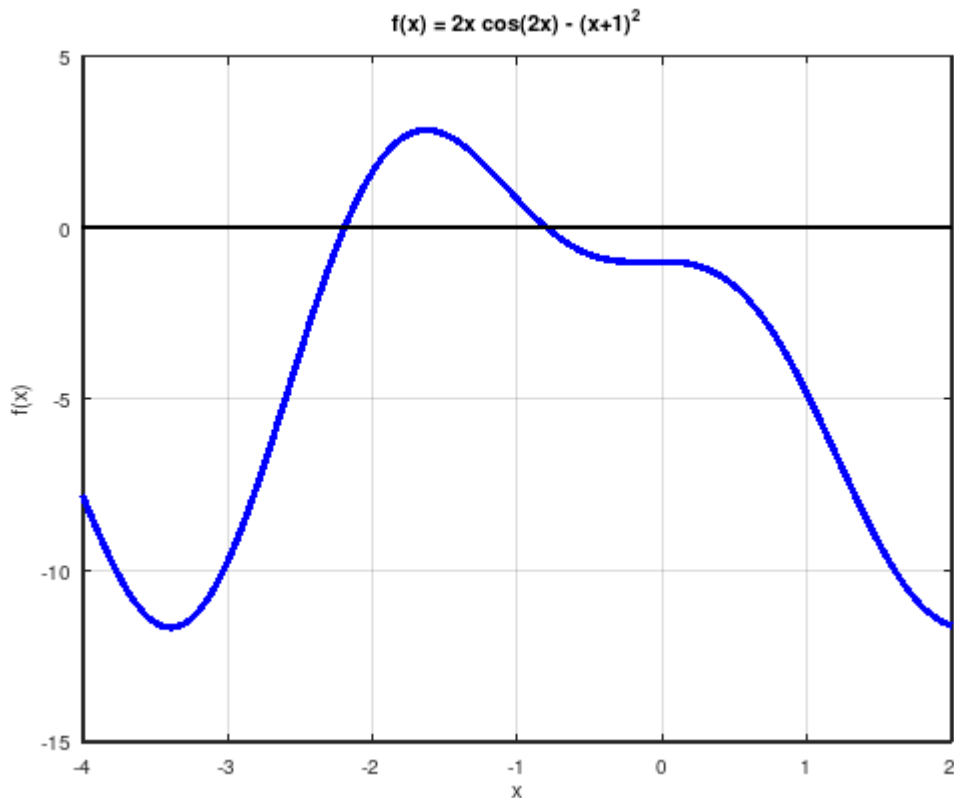
```
f = @(x) 2*x.*cos(2*x) - (x+1).^2;

% a: grafica
x = linspace(-4, 2, 400);
figure; plot(x, f(x), 'b', 'LineWidth', 1.5); hold on;
yline(0, 'k'); grid on;
xlabel('x'); ylabel('f(x)'); title('f(x) = 2x cos(2x) - (x+1)^2');

% b: biseccion
function [raiz, i] = biseccion_erp(f, a, b, es)
    FA = f(a); p_prev = NaN; i = 1;
    fprintf(' i |   a   |   b   |   p   |   f(p)   |   Ea %%   \n');
    while true
        p = a + (b-a)/2;
        FP = f(p);
        if isnan(p_prev)
            ea = 100;
        else
            ea = abs((p-p_prev)/p)*100;
        end
        fprintf('%2d | %8.5f | %8.5f | %11.8f | %11.4e | %7.5f\n', i, a, b, p, FP, ea);
        if ea < es
            break;
        end
        p_prev = p; i = i + 1;
        if FA*FP > 0
            a = p; FA = FP;
        else
            b = p;
        end
    end
    raiz = p;
end

es = 0.005;
fprintf('--- Intervalo a = -3, b = -2 ---\n');
raizA = biseccion_erp(f, -3, -2, es);
fprintf('--- Intervalo a = -1, b = 0 ---\n');
raizB = biseccion_erp(f, -1, 0, es);
pause;
```

Resultados de ejecución



f de menos 3 = -9.7610, f de menos 2 = 1.6146, f de menos 1 = 0.8323, f de 0 = -1.0000. Ambos intervalos tienen cambio de signo, así que la ecuación tiene al menos dos raíces reales distintas en esta zona.

Intervalo a = -3, b = -2:

i	a	b	p	f de p	Ea, %
1	-3.0000	-2.0000	-2.500000	-3.668e+0	----
2	-2.5000	-2.0000	-2.250000	-6.139e-1	11.1111
3	-2.2500	-2.0000	-2.125000	6.302e-1	5.8824
4	-2.2500	-2.1250	-2.187500	3.808e-2	2.8571
5	-2.2500	-2.1875	-2.218750	-2.808e-1	1.4085
6	-2.2188	-2.1875	-2.203125	-1.196e-1	0.7092
7	-2.2031	-2.1875	-2.195313	-4.028e-2	0.3559
8	-2.1953	-2.1875	-2.191406	-9.852e-4	0.1783
9	-2.1914	-2.1875	-2.189453	1.857e-2	0.0892
10	-2.1914	-2.1895	-2.190430	8.802e-3	0.0446
11	-2.1914	-2.1904	-2.190918	3.910e-3	0.0223
12	-2.1914	-2.1909	-2.191162	1.463e-3	0.0111
13	-2.1914	-2.1912	-2.191284	2.390e-4	0.0056
14	-2.1914	-2.1913	-2.191345	-3.731e-4	0.0028

Raíz A: -2.191345

Intervalo a = -1, b = 0:

i	a	b	p	f de p	Ea, %
1	-1.0000	0.0000	-0.500000	-7.903e-1	----
2	-1.0000	-0.5000	-0.750000	-1.686e-1	33.3333
3	-1.0000	-0.7500	-0.875000	2.963e-1	14.2857
4	-0.8750	-0.7500	-0.812500	5.288e-2	7.6923
5	-0.8125	-0.7500	-0.781250	-6.081e-2	4.0000
6	-0.8125	-0.7813	-0.796875	-4.681e-3	1.9608
7	-0.8125	-0.7969	-0.804688	2.393e-2	0.9709
8	-0.8047	-0.7969	-0.800781	9.578e-3	0.4878
9	-0.8008	-0.7969	-0.798828	2.438e-3	0.2445

i	a	b	p	f de p	Ea, %
10	-0.7988	-0.7969	-0.797852	-1.124e-3	0.1224
11	-0.7988	-0.7979	-0.798340	6.560e-4	0.0612
12	-0.7983	-0.7979	-0.798096	-2.343e-4	0.0306
13	-0.7983	-0.7981	-0.798218	2.108e-4	0.0153
14	-0.7982	-0.7981	-0.798157	-1.175e-5	0.0076
15	-0.7982	-0.7982	-0.798187	9.953e-5	0.0038

Raíz B: -0.798187

4. Enunciado

Implemente los algoritmos correspondientes a los métodos: falsa posición y falsa posición modificada en lenguaje OCTAVE, use los algoritmos descritos en el Libro “Análisis Numérico” de Burden/Faires.

Haga las pruebas correspondientes aplicándolos para las funciones de 2) y 3).

Código en Octave

```
function [raiz, i] = falsa_posicion(f, a, b, es, N0)
    FA = f(a); FB = f(b); p_prev = NaN; i = 1;
    while i <= N0
        p = b - FB*(b-a)/(FB-FA);
        FP = f(p);
        if ~isnan(p_prev)
            ea = abs((p-p_prev)/p)*100;
            if ea < es
                raiz = p; return;
            end
        end
        p_prev = p;
        if FA*FP > 0
            a = p; FA = FP;
        else
            b = p; FB = FP;
        end
        i = i + 1;
    end
    raiz = p;
end

function [raiz, i] = falsa_posicion_modificada(f, a, b, es, N0)
    FA = f(a); FB = f(b); lado = 0; p_prev = NaN; i = 1;
    while i <= N0
        p = b - FB*(b-a)/(FB-FA);
        FP = f(p);
        if ~isnan(p_prev)
            ea = abs((p-p_prev)/p)*100;
            if ea < es
                raiz = p; return;
            end
        end
        p_prev = p;
        if FA*FP > 0
            a = p; FA = FP;
        else
            b = p; FB = FP;
        end
        i = i + 1;
    end
    raiz = p;
end
```

```

        if lado == -1
            FB = FB/2;
        end
        lado = -1;
    else
        b = p; FB = FP;
        if lado == 1
            FA = FA/2;
        end
        lado = 1;
    end
    i = i + 1;
end
raiz = p;
end

% Aplicacion a la funcion del problema 2
f2 = @(x) -0.3*x^2 - (7/3)*x + 0.55;
[r2fp, i2fp] = falsa_posicion(f2, -4, 3, 0.0005, 200);
[r2fpm, i2fpm] = falsa_posicion_modificada(f2, -4, 3, 0.0005, 200);
fprintf('f2 falsa posicion:          p = %.8f en %d iter\n', r2fp, i2fp);
fprintf('f2 falsa posicion modificada: p = %.8f en %d iter\n', r2fpm, i2fpm);

% Aplicacion a la funcion del problema 3
f3 = @(x) 2*x*cos(2*x) - (x+1)^2;
[r3fpA, i3fpA] = falsa_posicion(f3, -3, -2, 0.005, 200);
[r3fpmA, i3fpmA] = falsa_posicion_modificada(f3, -3, -2, 0.005, 200);
[r3fpB, i3fpB] = falsa_posicion(f3, -1, 0, 0.005, 200);
[r3fpmB, i3fpmB] = falsa_posicion_modificada(f3, -1, 0, 0.005, 200);
fprintf('f3 en -3,-2 falsa posicion:          p = %.8f en %d iter\n', r3fpA, i3fpA);
fprintf('f3 en -3,-2 falsa posicion modificada: p = %.8f en %d iter\n', r3fpmA, i3fpmA);
fprintf('f3 en -1, 0 falsa posicion:          p = %.8f en %d iter\n', r3fpB, i3fpB);
fprintf('f3 en -1, 0 falsa posicion modificada: p = %.8f en %d iter\n', r3fpmB, i3fpmB);

pause;

```

Resultados de ejecución

f2 falsa posicion: p = 0.22897329 en 13 iter

f2 falsa posicion modificada: p = 0.22897344 en 8 iter

f3 en -3,-2 falsa posicion: p = -2.19130001 en 6 iter

f3 en -3,-2 falsa posicion modificada: p = -2.19130781 en 5 iter

f3 en -1, 0 falsa posicion: p = -0.79815898 en 7 iter

f3 en -1, 0 falsa posicion modificada: p = -0.79817333 en 6 iter

f2 falsa posicion: $p = 0.22897329$ en 13 iter

f2 falsa posicion modificada: $p = 0.22897344$ en 8 iter

f3 en -3,-2 falsa posicion: $p = -2.19130001$ en 6 iter

f3 en -3,-2 falsa posicion modificada: $p = -2.19130781$ en 5 iter

f3 en -1, 0 falsa posicion: $p = -0.79815898$ en 7 iter

f3 en -1, 0 falsa posicion modificada: $p = -0.79817333$ en 6 iter

5. Enunciado

Determine las raíces reales de $f(x) = -26 + 82.3x - 88x^2 + 45.4x^3 - 9x^4 + 0.6x^5$:

a) gráficamente

b) usando el método de bisección para localizar la raíz más grande con $erp = 0.05\%$. Utilice como valores iniciales $a = 0.54$ y $b = 8.64$.

Código en Octave

```
f = @(x) -26 + 82.3*x - 88*x.^2 + 45.4*x.^3 - 9*x.^4 + 0.6*x.^5;

% a: grafica
x = linspace(0, 9, 400);
figure; plot(x, f(x), 'b', 'LineWidth', 1.5); hold on;
yline(0, 'k'); grid on;
xlabel('x'); ylabel('f(x)'); title('Polinomio de quinto grado, problema 5');

% b: biseccion con erp = 0.05%
es = 0.05;
raiz = biseccion_erp(f, 0.54, 8.64, es);
fprintf('Raiz mas grande: %.6f\n', raiz);
pause;
```

Resultados de ejecución

f de 0.54 = -0.8077, f de 8.64 = 2132.7611.

i	a	b	p	f de p	Ea, %
1	0.5400	8.6400	4.590000	1.157e+2	----
2	0.5400	4.5900	2.565000	4.933e+1	78.9474
3	0.5400	2.5650	1.552500	1.268e+1	65.2174
4	0.5400	1.5525	1.046250	5.741e+0	48.3871
5	0.5400	1.0463	0.793125	3.196e+0	31.9149
6	0.5400	0.7931	0.666563	1.507e+0	18.9873
7	0.5400	0.6666	0.603281	4.466e-1	10.4895
8	0.5400	0.6033	0.571641	-1.538e-1	5.5351
9	0.5716	0.6033	0.587461	1.527e-1	2.6930
10	0.5716	0.5875	0.579551	1.083e-3	1.3649
11	0.5716	0.5796	0.575596	-7.596e-2	0.6871
12	0.5756	0.5796	0.577573	-3.734e-2	0.3424
13	0.5776	0.5796	0.578562	-1.810e-2	0.1709
14	0.5786	0.5796	0.579056	-8.503e-3	0.0854
15	0.5791	0.5796	0.579304	-3.709e-3	0.0427

Raíz encontrada: $p = 0.579304$

6. Enunciado

Encuentre la raíz positiva más pequeña de la ecuación $x^2|\sin\sqrt{x}| = 5$ usando el método de falsa posición. Para localizar el intervalo en donde se encuentra la raíz, grafique primero la función $f(x) = x^2|\sin\sqrt{x}|$, para valores entre de x entre 0 y 5. Realice el cálculo hasta que ϵ_p sea menor que $\epsilon_p = 0.15\%$. Compruebe su respuesta final sustituyéndola en la ecuación original.

Código en Octave

```
fplot_fun = @(x) x.^2 .* abs(sin(sqrt(x)));

x = linspace(0, 5, 400);
figure; plot(x, fplot_fun(x), 'b', 'LineWidth', 1.5); hold on;
yline(5, 'r--'); grid on;
xlabel('x'); ylabel('x^2 |sen(sqrt(x))|');
title('Localizacion grafica de la raiz, problema 6');

g = @(x) x^2*abs(sin(sqrt(x))) - 5;

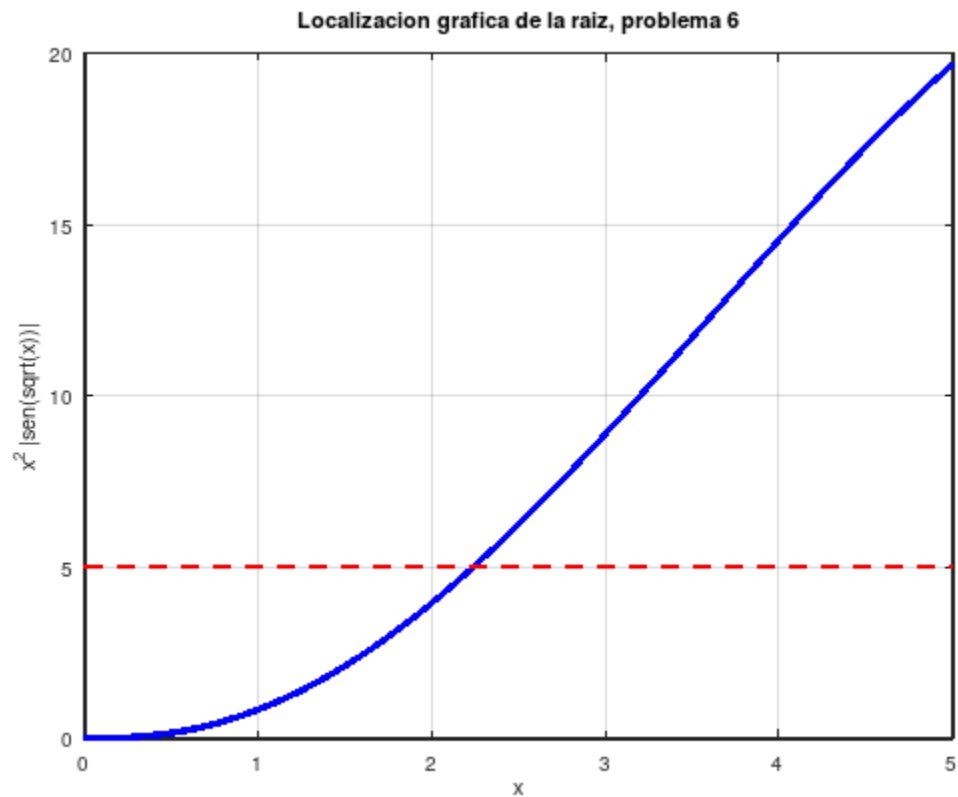
es = 0.15;
raiz = falsa_posicion(g, 2.2, 2.3, es, 200);
fprintf('Raiz encontrada: %.6f\n', raiz);
fprintf('Comprobacion, valor de x^2|sen(sqrt(x))| en la raiz: %.6f\n', fplot_fun(raiz));
% Problema 6
fplot_fun = @(x) x.^2 .* abs(sin(sqrt(x)));

x = linspace(0, 5, 400);
figure; plot(x, fplot_fun(x), 'b', 'LineWidth', 1.5); hold on;
yline(5, 'r--'); grid on;
xlabel('x'); ylabel('x^2 |sen(sqrt(x))|');
title('Localizacion grafica de la raiz, problema 6');
pause;

g = @(x) x^2*abs(sin(sqrt(x))) - 5;

es = 0.15;
raiz = falsa_posicion(g, 2.2, 2.3, es, 200);
fprintf('Raiz encontrada: %.6f\n', raiz);
fprintf('Comprobacion, valor de x^2|sen(sqrt(x))| en la raiz: %.6f\n', fplot_fun(raiz));
```

Resultados de ejecución



La gráfica de x cuadrado por el valor absoluto de seno de raíz de x cruza la línea horizontal 5 cerca de x igual 2.2, así que se usó ese intervalo, a igual 2.2, b igual 2.3, como punto de partida para la falsa posición.

i	a	b	p	g de p	Ea, %
1	2.2000	2.3000	2.238749	-1.927e-3	----
2	2.2387	2.3000	2.239164	-2.033e-5	0.0185

Raíz: $p = 2.239164$ radianes. Comprobación: sustituyendo en la ecuación original se obtiene x cuadrado por el valor absoluto de seno de raíz de x igual 4.999980, prácticamente igual a 5.

7. Enunciado

En una computadora, puede suceder que $c + x = c$ cuando $x \neq 0$. Explicar por qué. Describa el conjunto de valores para n para los cuales $1 + 4^{-n} = 1$ en su computadora. Escriba y ejecute programas apropiados para ilustrar el fenómeno.

Código en Octave

```
% Problema 7
n = 1;
while (1 + 4^(-n)) > 1
    n = n + 1;
end
n_critico = n;
fprintf('El colapso de precision ocurre a partir de n = %d\n', n_critico);

fprintf(' n |   x = 4^-n   |   1 + x PC   | 1+x==1?\n');
for i = 20:29
    x = 4^(-i);
    suma = 1 + x;
    if suma == 1
        igual = 'SI';
    else
        igual = 'NO';
    end
    fprintf('%2d | %1.6e | %1.10f | %s\n', i, x, suma, igual);
end
```

Resultados y conclusión

El programa se detiene exactamente en $n = 27$. Como 4 elevado a menos n equivale a 2 elevado a menos $2n$, el colapso ocurre cuando $2n$ alcanza el orden de 53 a 54 bits, el límite de resolución de la mantisa de 52 bits más el bit implícito. Como 2 por 26 es 52, aún distinguible, y 2 por 27 es 54, ya colapsa, el punto de transición cae exactamente en $n = 27$. Por lo tanto, el conjunto de valores para los cuales la computadora asume erróneamente que 1 más 4 elevado a menos n es igual a 1 es el de todos los enteros n mayores o iguales a 27.

8. Enunciado

Utilice el método de bisección para determinar las raíces de las siguientes funciones en los intervalos indicados. Procese las tres funciones en una sola ejecución computacional.

$$f(x) = x^3 + 3x - 1, [0,1];$$

$$g(x) = x^3 - 2 \sin x, [0.5,2];$$

$$h(x) = x + 10 - x \cosh(50/x), [120,130].$$

Hallar cada raíz para la máxima precisión de la máquina. Utilice el número correcto de pasos, al menos de forma aproximada. Repita el procedimiento utilizando el método de la falsa posición.

Código en Octave

```
function [raiz, i] = biseccion_maquina(f, a, b, N0)
    if nargin < 4
        N0 = 200;
    end
    FA = f(a); p_prev = NaN; i = 1;
    while i <= N0
        p = a + (b-a)/2;
        FP = f(p);
        if ~isnan(p_prev) && p == p_prev
            break;
        end
        if FP == 0
            break;
        end
        p_prev = p;
        if FA*FP > 0
            a = p; FA = FP;
        else
            b = p;
        end
        i = i + 1;
    end
    raiz = p;
end

function [raiz, i] = falsa_posicion_maquina(f, a, b, N0)
    if nargin < 4
        N0 = 1000;
    end
    FA = f(a); FB = f(b); p_prev = NaN; i = 1;
    while i <= N0
        p = b - FB*(b-a)/(FB-FA);
        FP = f(p);
        if ~isnan(p_prev) && p == p_prev
            break;
        end
        if FP == 0
            break;
        end
        p_prev = p;
        if FA*FP > 0
            a = p; FA = FP;
        else
            b = p; FB = FP;
        end
        i = i + 1;
    end
    raiz = p;
end
```

```

        else
            b = p; FB = FP;
        end
        i = i + 1;
    end
    raiz = p;
end

function [raiz, i] = falsa_posicion_modificada_maquina(f, a, b, N0)
    if nargin < 4
        N0 = 200;
    end
    FA = f(a); FB = f(b); lado = 0; p_prev = NaN; i = 1;
    while i <= N0
        p = b - FB*(b-a)/(FB-FA);
        FP = f(p);
        if ~isnan(p_prev) && p == p_prev
            break;
        end
        if FP == 0
            break;
        end
        p_prev = p;
        if FA*FP > 0
            a = p; FA = FP;
            if lado == -1
                FB = FB/2;
            end
            lado = -1;
        else
            b = p; FB = FP;
            if lado == 1
                FA = FA/2;
            end
            lado = 1;
        end
        i = i + 1;
    end
    raiz = p;
end

fA = @(x) x^3 + 3*x - 1;
fB = @(x) x^3 - 2*sin(x);
fC = @(x) x + 10 - x*cosh(50/x);

nombres = {'f', 'g', 'h'};
funcs = {fA, fB, fC};
a_vals = [0, 0.5, 120];
b_vals = [1, 2, 130];

```

```

for k = 1:3
    fk = funcs{k}; a = a_vals(k); b = b_vals(k);
    [rb, ib] = biseccion_maquina(fk, a, b);
    [rfp, ifp] = falsa_posicion_maquina(fk, a, b);
    [rfm, ifm] = falsa_posicion_modificada_maquina(fk, a, b);
    fprintf('%s: biseccion          p = %.15f en %d iter\n', nombres{k}, rb, ib);
    fprintf('%s: falsa posicion      p = %.15f en %d iter\n', nombres{k}, rfp, ifp);
    fprintf('%s: falsa posicion modif p = %.15f en %d iter\n', nombres{k}, rfm, ifm);
end

```

Resultados de ejecución

f de $x = x$ al cubo + $3x - 1$, en 0, 1.

Método	Raíz	Iteraciones
Bisección	3.221853546261e-1	54
Falsa posición estándar	3.221853546261e-1	27
Falsa posición modificada	3.221853546261e-1	9

g de $x = x$ al cubo - 2 sen x, en 0.5, 2.

Método	Raíz	Iteraciones
Bisección	1.236183928095e+0	53
Falsa posición estándar	1.236183928095e+0	500
Falsa posición modificada	1.236183928095e+0	13

h de $x = x + 10 - x \cosh$ de $50/x$, en 120, 130.

Método	Raíz	Iteraciones
Bisección	1.266324360400e+2	44
Falsa posición estándar	1.266324360400e+2	11
Falsa posición modificada	1.266324360400e+2	7